# Quantum Circuit Implementation and Resource Analysis of AIM2

Gyeongju Song, Kyungbae Jang, Seyoung Yoon,
Minwoo Lee, and Hwajeong Seo

Hansung University, South Korea
{thdrudwn98,starj1023,sebbang99,minunejip,hwajeong84}@gmail.com

**Abstract.** In this paper, we propose a quantum circuit implementation of AIM2. We apply optimization to reduce the circuit depth and introduce a method to reuse qubits by performing inverse operations in parallel. For all AIM2 variants (AIM2-I, AIM2-III, and AIM2-V), we design quantum circuits for $\mathsf{Mer}$, $\mathsf{Mer}^{-1}$, the linear layer, and feed-forward. We confirm that the $\mathsf{Mer}^{-1}$ operation dominates the overall cost of AIM2 constructions. Compared to the previous version of AIM, AIM2 requires significantly more quantum resources since it introduces $\mathsf{Mer}^{-1}$ before the linear layer. Based on the proposed circuits, we estimate the cost of Grover's algorithm for key search and compare it with the NIST estimates for AES complexity. As a result, AIM2-I, AIM2-III, and AIM2-V achieve the post-quantum security levels of Level-1, Level-3, and Level-5, respectively.

**Keywords:** Quantum Implementation · Quantum Computing · AIM2 · AIMer · Grover's algorithm

## 1 Introduction

Quantum computers pose a significant threat to modern cryptography, attracting growing attention from the research community. Shor's algorithm [15,16] can solve the fundamental problems underlying public-key cryptosystems, such as integer factorization and discrete logarithms, in polynomial time, thereby undermining their security foundations. On the other hand, Grover's algorithm [4] reduces the number of iterations required for brute-force attacks and pre-image attacks against symmetric-key ciphers and hash functions from $N$ to $\sqrt{N}$, effectively decreasing the attack cost to the square root of the classical complexity. Consequently, symmetric-key ciphers must typically double their key size to maintain a reasonable level of security against quantum adversaries.

However, there is a clear distinction between theoretical complexity and practical attack feasibility. While Grover's algorithm theoretically reduces the security level of key search to the square root, actual quantum key recovery requires extremely large iteration counts and substantial quantum resources, rendering real-world attacks still infeasible. Given that current quantum hardware cannot yet sustain such massive repetitions, estimating the quantum resources required for potential attacks provides valuable insight into the security lifetime

of symmetric-key ciphers. In particular, if the quantum resources required to attack a given cipher remain excessively large, the cipher may be regarded as secure against quantum adversaries without increasing its key length. In this context, the post-quantum security requirements defined by the U.S. National Institute of Standards and Technology (NIST) [13,14] serve as an important reference, where post-quantum security levels (Level-1 to Level-5) are introduced to evaluate cryptographic strength under quantum attacks.

In this work, we present the first quantum cryptanalysis of AIM2, the symmetric primitive employed in AIMer, a digital signature scheme selected as one of the final algorithms in the Korea Post-Quantum Cryptography (KPQC) competition (https://www.kpqc.or.kr/competition_02.html, accessed on 23 September 2025). AIMer version 2 [11] updates its predecessor version 1 [10] by adopting AIM2 as the new internal symmetric primitive. While AIM (used in AIMer ver 1) has previously been implemented as a quantum circuit in [7], no prior work has addressed the quantum design and optimization of AIM2.

Our contributions are summarized as follows:

- We propose the first quantum circuit implementation of AIM2.
- Our optimization prioritizes reducing the circuit depth, followed by minimizing the number of qubits.
- Based on the optimized circuits, we estimate the quantum resources required to run Grover's algorithm and evaluate the post-quantum security level of AIM2 according to NIST's AES criteria.

## 2 Bacground

### 2.1 Quantum Computing

Unlike classical computers that use bits with values of 0 or 1, quantum computers use qubits, which can represent both 0 and 1 simultaneously through superposition. This property enables quantum computers to process many possibilities in parallel and solve certain problems much faster than classical computers. A more detailed discussion of quantum computing is provided in [12]. Quantum circuits compute by applying quantum gates to qubits. We measure circuit cost primarily by its width (number of qubits), depth (number of sequential gate layers), and gate counts. In this work, to implement AIM2 reversibly we use the standard gate set X, CNOT, Toffoli.

- **X(NOT) gate**: Flips a single qubit: $|x\rangle \leftarrow |x \oplus 1\rangle$
- **CNOT (Controlled-NOT)**: With control $x$ and target $y$, flips the target only if $x = 1$: $|x, y\rangle \leftarrow |x,\ x \oplus y\rangle$ (i.e., XOR on the target).
- **Toffoli (CCNOT)**: With controls $x, y$ and target $z$, flips the target only if $x = y = 1$: $|x, y, z\rangle \leftarrow |x, y,\ z \oplus (x \wedge y)\rangle$.

Using only the X, CNOT, and Toffoli gates we can realize the classical operations NOT, XOR, and AND in a reversible form, so a block-cipher round can be built without loss of information. In practice, Toffoli gates cost the most.

We adopt a common decomposition in which one Toffoli ≈ 7 T gates + 8 Clifford gates, with T-depth 4 and full depth 8 [1]; other decompositions can be accommodated by adjusting these constants.

### 2.2 Grover's Algorithm

Grover's algorithm is a quantum search procedure that reduces the cost of brute-force attacks on symmetric ciphers. For a $k$-bit key, classical search requires $O(2^k)$ operations, while Grover's algorithm achieves the task with $O(\sqrt{2^k})$ evaluations, effectively halving the security level in bits. For example, AES-128, which resists $2^{128}$ classical operations, can be attacked with about $2^{64}$ quantum steps.

*1. Input preparation.* Apply Hadamard gates to $k$ qubits to create an equal superposition of all keys:

$$|\psi\rangle = H^{\otimes k} |0\rangle^{\otimes k} = \frac{1}{2^{k/2}} \sum_{x=0}^{2^k-1} |x\rangle . \tag{1}$$

*2. Oracle.* The oracle checks whether a candidate key encrypts the known plaintext to the target ciphertext and flips the phase if it does:

$$f(x) = \begin{cases} 1 & \text{if } \ Enc_x(P) = C, \\ 0 & \text{otherwise,} \end{cases} \qquad U_f |x\rangle = (-1)^{f(x)} |x\rangle . \tag{2}$$

*3. Diffusion.* A diffusion operator amplifies the marked state by inversion about the mean, increasing the probability of measuring the correct key.

## 3 NIST Post-quantum Security

To evaluate the security of a cipher against quantum attacks, NIST specifies security bounds for the cipher [13,14]:

- Level 1: Resource requirements for the attack are similar to those for breaking AES-128 ($2^{170} \rightarrow \mathbf{2^{157}}$).
- Level 3: Resource requirements for the attack are similar to those for breaking AES-192 ($2^{233} \rightarrow \mathbf{2^{221}}$).
- Level 5: Resource requirements for the attack are similar to those for breaking AES-256 ($2^{298} \rightarrow \mathbf{2^{285}}$).

According to NIST's estimation, which is based on the quantum cost analysis of Grover's key search for AES variants presented in PQCrypto'16 [3], the quantum attack complexities for security Levels 1, 3 and 5 (corresponding to different key sizes) are approximately $2^{170}$, $2^{233}$ and $2^{298}$ [13], respectively, where the values represent (the total number of gates) × (the depth of Grover's search). More

recently, following the updated results reported in Eurocrypt'20 [8], NIST revised these estimates, lowering the quantum attack costs for AES-128, AES-192, and AES-256 to $2^{157}$, $2^{221}$, and $2^{285}$ [14], respectively.

While Grover's algorithm provides a theoretical square-root reduction in brute-force cost, its realization requires quantum circuits of prohibitive depth, far beyond the reach of current technology. To formalize this limitation, NIST introduced the notion of *MAXDEPTH*.

## 4 AIM2

AIM2 has been proposed as a tweakable one-way function that provides multi-target one-wayness, making it suitable for signature schemes [11]. Its construction combines non-linear components, a linear layer, and an additional constant injection.

The non-linear part relies on Mersenne-type S-boxes defined by exponentiation of the form $x^{2^e-1}$ together with their inverses. The exponents $e$ are chosen such that $\gcd(e, n) = 1$, ensuring that the inverse exponent is well-defined. This choice further guarantees that the resulting inverse S-box can be expressed by quadratic equations, which strengthens the one-wayness of the primitive.

The linear layer consists of three components: (i) an `AddConst` operation that injects fixed constants $\gamma_1, \ldots, \gamma_\ell$, (ii) an affine transformation based on a random invertible matrix and vector derived from an extendable-output function (XOF) with input iv, and (iii) a feed-forward step where the input is added back to the output, making the overall mapping non-invertible.

The structure of AIM2 used in this work is illustrated in Figure 1, where the example corresponds to AIM2-V with $\ell = 3$. In this setting, the input pt represents the secret key of the signature scheme, while the pair (iv, ct) can be seen as the corresponding public key.

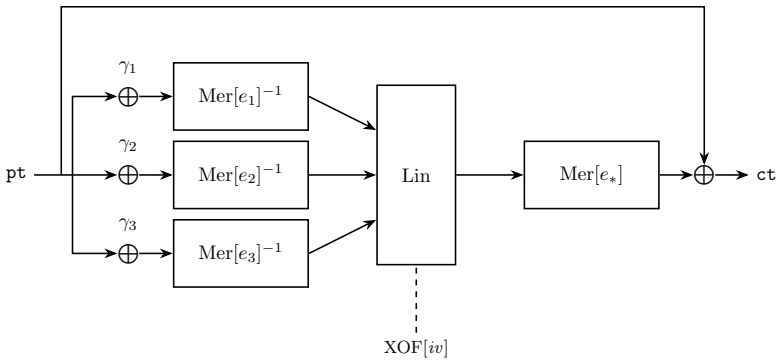


Fig. 1: Structure of the AIM2-V one-way function ($\ell = 3$).

Recommended parameter sets are provided for security levels $\lambda \in \{128, 192, 256\}$. Each variant—AIM2-I, AIM2-III, and AIM2-V—is defined by different input/output sizes $n$, tuples of exponents $(e_1, \ldots, e_\ell, e_*)$, and irreducible polynomials for the field $\mathbb{F}_{2^n}$. For example, AIM2-V adopts $n = 256$, $\ell = 3$, and exponents $(11, 141, 7; 3)$, with $\mathbb{F}_{2^{256}}$ constructed using the polynomial $X^{256} + X^{10} + X^5 + X^2 + 1$.

## 5 Quantum circuit for AIM2

In this paper, we propose a quantum circuit implementation of AIM2. Our first optimization focuses on reducing the circuit depth, and the second aims to minimize the number of qubits. Among the AIM2 components, the one that consumes the largest amount of quantum resources is $\mathrm{Mer}^{-1}(e)$, followed by $\mathrm{Mer}(e)$. Specifically, $\mathrm{Mer}^{-1}(e)$ corresponds to exponentiation by the multiplicative inverse of $(2^e - 1)$ modulo $(2^n - 1)$, which involves a large number of squaring and multiplication operations. In contrast, $\mathrm{Mer}(e)$ is defined as exponentiation by $(2^e - 1)$, and although it is also resource-intensive, its cost is smaller compared to $\mathrm{Mer}^{-1}(e)$.

Before describing the quantum circuit for AIM2, we first outline the multiplication and squaring operations in the binary field $\mathbb{F}_{2^n}$.

For multiplication, we use the Karatsuba algorithm proposed in [6]. This method recursively decomposes the product into smaller blocks until the size becomes $1 \times 1$. The main idea is to allocate additional ancilla qubits to prepare all intermediate operands in advance so that the partial multiplications can be performed in parallel. As a result, the overall Toffoli depth of the multiplication is reduced to 1 for any field size, which is lower than other binary multiplication techniques[9,17,2]. The approach in [6] requires many ancilla qubits to achieve low depth. However, in AIM2 the multiplications inside $\mathrm{Mer}^{-1}(e)$ and $\mathrm{Mer}(e)$ are executed sequentially rather than in parallel. This allows the same ancilla qubits to be reused across multiple multiplications, thereby reducing the number of qubits while maintaining a low depth. For squaring, the implementation is simpler and less costly than multiplication. Since no cross products are required, the operation can be realized using only CNOT gates followed by modular reduction. Table 1 provides the estimated quantum resources for multiplication and squaring in $\mathbb{F}_{2^{128}} = (x^{128} + x^7 + x^2 + x + 1)$, $\mathbb{F}_{2^{192}} = (x^{192} + x^7 + x^2 + x + 1)$, and $\mathbb{F}_{2^{256}} = (x^{256} + x^{10} + x^5 + x^2 + 1)$.

Modular reduction is a linear operation, and while an in-place implementation via PLU decomposition is possible, [11] performs the reduction directly for simplicity. In this case, only a small number of additional ancilla qubits (3–5) are required in addition to the input qubits. We also adopt the simplified modular reduction described in [11].

### 5.1 $\mathrm{Mer}^{-1}$

Before the linear layer, the $\mathrm{Mer}^{-1}$ operation is performed as a sequence of squaring and multiplication, which incurs the majority of the computational cost in

Table 1: Quantum resources required for multiplication and squaring in $\mathbb{F}_{2^{128}}$, $\mathbb{F}_{2^{192}}$, and $\mathbb{F}_{2^{256}}$.

| Field size $2^n$ | Operation | #CNOT | #1qCliff | $\#T$ | $T$-depth | #Qubit | Full depth |
|---|---|---|---|---|---|---|---|
| $n = 128$ | mul | 29867 | 4374 | 15309 | 4 | 6561 | 78 |
| | squ | 205 | - | - | - | 131 | 127 |
| $n = 192$ | mul | 85577 | 10206 | 35721 | 4 | 15309 | 94 |
| | squ | 301 | - | - | - | 195 | 196 |
| $n = 256$ | mul | 115558 | 13122 | 45927 | 4 | 19683 | 180 |
| | squ | 401 | - | - | - | 261 | 253 |

AIM2. In this paper, we focus on $\mathrm{Mer}^{-1}(49)$ used in AIM2-I. Since this operation involves many multiplications and squarings, the corresponding pseudocode is divided into Algorithm 1 for readability.

$\mathrm{Mer}^{-1}(49)$ consists of 24 multiplications and 24 squarings. Some qubits are reset and reused via inverse operations. Because these inverse operations are executed in parallel with the forward computations, they do not increase the overall circuit depth while restoring qubits to the clean state. In addition, certain sequential steps are parallelized to further improve efficiency.

The pseudocode notation `CNOT(`$a_{[0:127]}$`,` $b_{[0:127]}$`)` indicates bitwise CNOT operations between $a$ and $b$, which correspond to 128 parallel CNOT gates and therefore contribute depth 1. The `CleanAncilla` operation initializes ancilla qubits used in multiplications (e.g., 4118 qubit) with negligible overhead, as discussed in Section 3.3 in [6].

As another optimization, the values $t1$ and $table\_5$ computed in lines 4–10 of Algorithm 1 are no longer required after line 31. Thus, they are reset in parallel and reused in the subsequent linear layer. Moreover, the multiplications at lines 26 and 29 share the same input $table\_b$ and would normally be sequential. By applying fan-out to $t_1^{copy}$, these multiplications can be executed in parallel. After completion, $t_1^{copy}$ is reset and reused at line 32.

These optimization techniques apply not only to $\mathrm{Mer}^{-1}(49)$ but also to $\mathrm{Mer}^{-1}(91)$ and other variants. However, this paper focuses on AIM2-I, and we omit the implementation details of AIM2-III and AIM2-V. Table 2 shows the estimated quantum resources of the $\mathrm{Mer}^{-1}$ functions used in AIM2-I, AIM2-III, and AIM2-V.

Table 2: Quantum resources required for the $\mathsf{Mer}^{-1}$ of AIM2.

| Cipher | Component | #CNOT | #$T$ | #1qCliff | #Qubit | Full depth |
|---|---|---|---|---|---|---|
| AIM2-I | $\mathsf{Mer}^{-1}(49)$ | 910,893 | 367,416 | 104,976 | 57,627 | 11,053 |
| | $\mathsf{Mer}^{-1}(91)$ | 925,597 | 367,416 | 104,976 | 57,372 | 11,154 |
| AIM2-III | $\mathsf{Mer}^{-1}(17)$ | 1,718,007 | 714,420 | 204,120 | 113,607 | 37,970 |
| | $\mathsf{Mer}^{-1}(47)$ | 2,409,706 | 1,107,351 | 316,386 | 316,386 | 39,529 |
| AIM2-V | $\mathsf{Mer}^{-1}(11)$ | 2,600,661 | 1,010,394 | 288,684 | 159,507 | 67,610 |
| | $\mathsf{Mer}^{-1}(141)$ | 2,716,910 | 1,056,321 | 301,806 | 166,314 | 67,555 |
| | $\mathsf{Mer}^{-1}(7)$ | 2,049,454 | 872,613 | 249,318 | 139,824 | 67,220 |

**Algorithm 1** Quantum circuit implementation of $\mathsf{Mer}^{-1}(49)$.

**Input:** $x$
**Output:** $out$, $anc$, $t1$, $table\_5$

// Allocate ancilla qubits for Mul
1: $anc \leftarrow$ allocate 4118 ancilla qubits

// Compute $\mathsf{Mer}^{-1}(49)$
2: table_d $\leftarrow$ allocate new 128 qubits
3: $t_1 \leftarrow$ allocate new 128 qubits

// $t_1 = x^4$
4: CNOT($x_{[0:127]}$, table_d$_{[0:127]}$)
5: table_d $\leftarrow$ Squaring(table_d)
6: CNOT(table_d$_{[0:127]}$, $t_{1[0:127]}$)
7: $t_1 \leftarrow$ Squaring($t_1$)

// table_5 $= x^5$
8: table_5 $\leftarrow$ Mul($t_1, x, anc$)
9: table_5 $\leftarrow$ Reduction(table_5)
10: $anc \leftarrow$ CleanAncilla($t_1, x, anc$)

// table_6 $= x^6$
11: table_6 $\leftarrow$ Mul(table_5, $x, anc$)
12: table_6 $\leftarrow$ Reduction(table_6)
13: $anc \leftarrow$ CleanAncilla(table_5, $x, anc$)

// table_a $= x^{10}$
14: CNOT(table_5$_{[0:127]}$, table_a$_{[0:127]}$)
15: table_a $\leftarrow$ Squaring(table_a)

// table_b $= x^{11}$
16: table_b $\leftarrow$ Mul(table_a, $x, anc$)
17: table_b $\leftarrow$ Reduction(table_b)
18: $anc \leftarrow$ CleanAncilla(table_a, $x, anc$)

// table_d $= x^{13}$
19: table_d $\leftarrow$ Mul(table_b, table_d, $anc$)
20: table_d $\leftarrow$ Reduction(table_d)
21: $anc \leftarrow$ CleanAncilla(table_b, table_d, $anc$)

// table_b $= x^{0xb6}$, table_5 $= x^{0xb5}$
22: **for** $i = 1$ to 4 **do**
23: table_b $\leftarrow$ Squaring(table_b)
24: $t_1^{\text{copy}} \leftarrow$ allocate new 128 qubits
25: CNOT(table_b, $t_1^{\text{copy}}$)
26: table_b_2 $\leftarrow$ Mul(table_b, table_6, $anc$)
27: table_b_2 $\leftarrow$ Reduction(table_b_2)
28: $anc \leftarrow$ CleanAncilla(table_b, table_6, $anc$)
29: table_5_2 $\leftarrow$ Mul($t_1^{\text{copy}}$, table_5, $anc$)
30: table_5_2 $\leftarrow$ Reduction(table_5_2)
31: $anc \leftarrow$ CleanAncilla($t_1^{\text{copy}}$, table_5, $anc$)

Inverse operation for lines 4-10 (Parallel)

// $t_1 = x^{0xb6b6}$
32: CNOT(table_b_2, $t_1^{\text{copy}}$)
33: **for** $i = 1$ to 8 **do**
34: $t_1^{\text{copy}} \leftarrow$ Squaring($t_1^{\text{copy}}$)
35: $t_{1\_2} \leftarrow$ Mul($t_1^{\text{copy}}$, table_b_2, $anc$)
36: $t_{1\_2} \leftarrow$ Reduction($t_{1\_2}$)
37: $anc \leftarrow$ CleanAncilla($t_1^{\text{copy}}$, table_b_2, $anc$)

// $t_1 = x^{0xb6b6d}$
38: **for** $i = 1$ to 4 **do**
39: $t_{1\_2} \leftarrow$ Squaring($t_{1\_2}$)
40: $t_{1\_3} \leftarrow$ Mul($t_{1\_2}$, table_d, $anc$)
41: $t_{1\_3} \leftarrow$ Reduction($t_{1\_3}$)
42: $anc \leftarrow$ CleanAncilla($t_{1\_2}$, table_d, $anc$)

// $t_1 = x^{0xb6b6d6}$
43: **for** $i = 1$ to 4 **do**
44: $t_{1\_3} \leftarrow$ Squaring($t_{1\_3}$)
45: $t_{1\_4} \leftarrow$ Mul($t_{1\_3}$, table_6, $anc$)
46: $t_{1\_4} \leftarrow$ Reduction($t_{1\_4}$)
47: $anc \leftarrow$ CleanAncilla($t_{1\_3}$, table_6, $anc$)

// $t_2 = x^{0xb6b6d6d}$
48: **for** $i = 1$ to 4 **do**
49: $t_{1\_4} \leftarrow$ Squaring($t_{1\_4}$)
50: $t_2 \leftarrow$ Mul($t_{1\_4}$, table_d, $anc$)
51: $t_2 \leftarrow$ Reduction($t_2$)
52: $anc \leftarrow$ CleanAncilla($t_{1\_4}$, table_d, $anc$)

**Algorithm 1** Quantum circuit implementation of $\mathsf{Mer}^{-1}(49)$. (continued)

// $t_1 = x^{0xb6b6d6d6}$
CNOT($t_2$, $t_1^{\mathrm{copy}}$)
**for** $i = 1$ to 4 **do**
    $t_2 \leftarrow \mathsf{Squaring}(t_2)$
$t_{1\_5} \leftarrow \mathsf{Mul}(t_2, \mathsf{table\_6}, anc)$
$t_{1\_5} \leftarrow \mathsf{Reduction}(t_{1\_5})$
$anc \leftarrow \mathsf{CleanAncilla}(t_2, \mathsf{table\_6}, anc)$

// $t_1 = x^{0xb6b6d6d6d6}$
**for** $i = 1$ to 4 **do**
    $t_{1\_5} \leftarrow \mathsf{Squaring}(t_{1\_5})$
$t_{1\_6} \leftarrow \mathsf{Mul}(t_{1\_5}, \mathsf{table\_d}, anc)$
$t_{1\_6} \leftarrow \mathsf{Reduction}(t_{1\_6})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_5}, \mathsf{table\_d}, anc)$

// $t_1 = x^{0xb6b6d6d6d6da}$
**for** $i = 1$ to 4 **do**
    $t_{1\_6} \leftarrow \mathsf{Squaring}(t_{1\_6})$
$t_{1\_7} \leftarrow \mathsf{Mul}(t_{1\_6}, \mathsf{table\_a}, anc)$
$t_{1\_7} \leftarrow \mathsf{Reduction}(t_{1\_7})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_6}, \mathsf{table\_a}, anc)$

// $t_1 = x^{0xb6b6d6d6d6dad}$
**for** $i = 1$ to 4 **do**
    $t_{1\_7} \leftarrow \mathsf{Squaring}(t_{1\_7})$
$t_{1\_8} \leftarrow \mathsf{Mul}(t_{1\_7}, \mathsf{table\_d}, anc)$
$t_{1\_8} \leftarrow \mathsf{Reduction}(t_{1\_8})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_7}, \mathsf{table\_d}, anc)$

// $t_1 = x^{0xb6b6d6d6d6dadb5}$
**for** $i = 1$ to 8 **do**
    $t_{1\_8} \leftarrow \mathsf{Squaring}(t_{1\_8})$
$t_{1\_9} \leftarrow \mathsf{Mul}(t_{1\_8}, \mathsf{table\_5\_2}, anc)$
$t_{1\_9} \leftarrow \mathsf{Reduction}(t_{1\_9})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_8}, \mathsf{table\_5\_2}, anc)$

// $t_1 = x^{0xb6b6d6d6d6dadb5b5}$
**for** $i = 1$ to 8 **do**
    $t_{1\_9} \leftarrow \mathsf{Squaring}(t_{1\_9})$
$t_{1\_10} \leftarrow \mathsf{Mul}(t_{1\_9}, \mathsf{table\_5\_2}, anc)$
$t_{1\_10} \leftarrow \mathsf{Reduction}(t_{1\_10})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_9}, \mathsf{table\_5\_2}, anc)$

// $t_1 = x^{0xb6b6d6d6d6dadb5b5b6}$
**for** $i = 1$ to 8 **do**
    $t_{1\_10} \leftarrow \mathsf{Squaring}(t_{1\_10})$
$t_{1\_11} \leftarrow \mathsf{Mul}(t_{1\_10}, \mathsf{table\_b\_2}, anc)$
$t_{1\_11} \leftarrow \mathsf{Reduction}(t_{1\_11})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_10}, \mathsf{table\_b\_2}, anc)$

// $t_1 = x^{0xb6b6d6d6d6dadb5b5b6b6d}$
**for** $i = 1$ to 28 **do**
    $t_{1\_11} \leftarrow \mathsf{Squaring}(t_{1\_11})$
$t_{1\_12} \leftarrow \mathsf{Mul}(t_{1\_11}, t_1^{\mathrm{copy}}, anc)$
$t_{1\_12} \leftarrow \mathsf{Reduction}(t_{1\_12})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_11}, t_1^{\mathrm{copy}}, anc)$

// $t_1 = x^{0xb6b6d6d6d6dadb5b5b6b6b6d6da}$
**for** $i = 1$ to 4 **do**
    $t_{1\_12} \leftarrow \mathsf{Squaring}(t_{1\_12})$
$t_{1\_13} \leftarrow \mathsf{Mul}(t_{1\_12}, \mathsf{table\_a}, anc)$
$t_{1\_13} \leftarrow \mathsf{Reduction}(t_{1\_13})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_12}, \mathsf{table\_a}, anc)$

// $t_1 = x^{0xb6b6d6d6d6dadb5b5b6b6b6d6dad}$
**for** $i = 1$ to 4 **do**
    $t_{1\_13} \leftarrow \mathsf{Squaring}(t_{1\_13})$
$t_{1\_14} \leftarrow \mathsf{Mul}(t_{1\_13}, \mathsf{table\_d}, anc)$
$t_{1\_14} \leftarrow \mathsf{Reduction}(t_{1\_14})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_13}, \mathsf{table\_d}, anc)$

// $t_1 = x^{0xb6b6d6d6d6dadb5b5b6b6b6d6dada}$
**for** $i = 1$ to 4 **do**
    $t_{1\_14} \leftarrow \mathsf{Squaring}(t_{1\_14})$
$t_{1\_15} \leftarrow \mathsf{Mul}(t_{1\_14}, \mathsf{table\_a}, anc)$
$t_{1\_15} \leftarrow \mathsf{Reduction}(t_{1\_15})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_14}, \mathsf{table\_a}, anc)$

// $t_1 = x^{0xb6b6d6d6d6dadb5b5b6b6b6d6dadad}$
**for** $i = 1$ to 4 **do**
    $t_{1\_15} \leftarrow \mathsf{Squaring}(t_{1\_15})$
$t_{1\_16} \leftarrow \mathsf{Mul}(t_{1\_15}, \mathsf{table\_d}, anc)$
$t_{1\_16} \leftarrow \mathsf{Reduction}(t_{1\_16})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_15}, \mathsf{table\_d}, anc)$

// $t_1 = x^{0xb6b6d6d6d6dadb5b5b6b6b6d6dadadb5}$
**for** $i = 1$ to 8 **do**
    $t_{1\_16} \leftarrow \mathsf{Squaring}(t_{1\_16})$
$t_{1\_17} \leftarrow \mathsf{Mul}(t_{1\_16}, \mathsf{table\_5\_2}, anc)$
$t_{1\_17} \leftarrow \mathsf{Reduction}(t_{1\_17})$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_16}, \mathsf{table\_5\_2}, anc)$

// $t_1 = x^{0xb6b6d6d6d6dadb5b5b6b6b6d6dadadb5b5}$
**for** $i = 1$ to 8 **do**
    $t_{1\_17} \leftarrow \mathsf{Squaring}(t_{1\_17})$

$out \leftarrow \mathsf{Mul}(t_{1\_17}, \mathsf{table\_5\_2}, anc)$
$out \leftarrow \mathsf{Reduction}(out)$
$anc \leftarrow \mathsf{CleanAncilla}(t_{1\_16}, \mathsf{table\_5\_2}, anc)$

**return** $out$, $anc$, $t1$, $table\_5$

### 5.2 Mer

The Mer operation consists of multiple multiplications and squarings, as presented in Algorithm 2. For simplicity, we describe Mer based on Mer(3) used in AIM2-I and AIM2-V. This function is performed after the linear operation, and Table 3 presents the resource estimation results for Mer.

Table 3: Quantum resources required for the Mer of AIM2.

| Cipher | Component | #CNOT | #$T$ | #1qCliff | #Qubit | Full depth |
|---|---|---|---|---|---|---|
| AIM2-I | Mer(3) | 68,508 | 30,618 | 8,748 | 8,754 | 410 |
| AIM2-III | Mer(5) | 240,325 | 107,163 | 30,618 | 25,911 | 1,061 |
| AIM2-V | Mer(3) | 206,954 | 91,854 | 206,954 | 26,254 | 855 |

**Algorithm 2** Quantum circuit implementation of Mer (3).

**Input:** *input*
//Compute Mer(3)
$t1 \leftarrow$ allocate new 128 qubits
CNOT($input_{[0:128]}$, $t1_{0:127}$)

$//t1 = a^{(2^2-1)}$
$t1 \leftarrow$ Squaring($t1$)

$t_{1\_2} \leftarrow$ Mul($t1$, *input*, *anc*)
$t_{1\_2} \leftarrow$ Reduction($t_{1\_2}$)
$anc \leftarrow$ CleanAncilla($t1$, *input*, *anc*)

$//out = a^{(2^3-1)}$
$t_{1\_2} \leftarrow$ Squaring($t_{1\_2}$)
$out \leftarrow$ Mul($t_{1\_2}$, *input*, *anc*)
$out \leftarrow$ Reduction(*out*)
$anc \leftarrow$ CleanAncilla($t_{1\_2}$, *input*, *anc*)

**return** *out*, *anc*

### 5.3 Linear layer

The Linear Layer consists of multiplying the input vector by a predefined binary matrix. This matrix is derived from the initial vector (hash input) using SHAKE-128 for AIM-I and SHAKE-256 for AIM-III and AIM-V, and since it is

publicly available, it can be treated as a constant. Therefore, the operation is a classical(matrix vector)-quantum rather than a quantum-quantum transformation, enabling efficient quantum circuit design based on classically precomputed matrices.

Among various approaches to implement matrix-vector multiplication in quantum circuits, [7] adopted an out-of-place design to optimize the depth. This method allocates fresh output qubits and performs XOR operations (CNOT gates) between the input and output vectors. Although it requires additional qubits compared to the in-place approach, it offers the advantage of reduced depth. Moreover, since the input vector is public, some operations can be replaced by applying X gates instead of CNOT gates, enabling further optimization.

In this work, we adopt a similar out-of-place Linear Layer design following the approach of [7]. Table 4 presents the estimated quantum resources for the Linear Layer. The CNOT gates depends on the number of ones in the matrix generated by SHAKE-128 or SHAKE-256. As a result, the count of the CNOT gate varies depending on the specific output matrices; thus, the number of CNOT is provided only for reference. The #Qubit denotes the total number of qubits required for the Linear Layer, and the values in parentheses indicate qubits reused from previous operations. In AIM2-I and AIM2-III, both the L-matrix and U-matrix sets consist of two matrices, represented as matrix_L=$[L_0, L_1]$ and matrix_U=$[U_0, U_1]$. In AIM2-V, each matrix set is extended to three, represented as matrix_L=$[L_0, L_1, L_2]$ and matrix_U=$[U_0, U_1, U_2]$. Consequently, AIM2-I and AIM2-III employ a total of four matrix arrays, while AIM2-V uses six. Since the number of ancilla qubits increases proportionally to the number of matrix arrays, AIM2-I and AIM2-III require $2n$ input qubits and $4n$ ancilla qubits, whereas AIM2-V requires $3n$ input qubits and $6n$ ancilla qubits.

Table 4: Quantum resources required for the LinearLayer in AIM2.

| LinearLayer | #CNOT | #Qubit | Full depth |
|---|---|---|---|
| AIM2-I ($n = 128$) | 16,858 | 768 (512) | 302 |
| AIM2-III ($n = 192$) | 37,607 | 1,152 (768) | 457 |
| AIM2-V ($n = 256$) | 99,800 | 2,304 (1,536) | 614 |

Finally, the FeedForward step requires XOR operations between the input and the output, for which CNOT gates are employed. Since the proposed design is based on the Karatsuba multiplication [6], a large number of ancilla qubits are required. However, in exchange for this qubit overhead, the overall circuit is designed to maintain a low full depth (see Table 5).

## 6 Evaluation

In this section, we discuss the post-quantum security of AIM2 based on its estimated quantum resources.

First, we compare the resource requirements of AIM and AIM2. Table 5 presents this comparison. The AIMer improved the scheme by replacing the internal tweakable one-way function AIM with AIM2. While AIM applies the Mer operation both before and after the linear layer, AIM2 uses $\text{Mer}^{-1}$ before the linear layer and Mer afterwards. Since $\text{Mer}^{-1}$ is significantly more costly than Mer, AIM2 circuits require more quantum resources than AIM circuits [7].

Table 5: Estimated quantum resources for the AIM2 quantum circuits.

| Cipher | #CNOT | #1qCliff | $\#T$ | #Qubit | Full depth | $FD \times M$ |
|---|---|---|---|---|---|---|
| AIM-I [7] | 358,754 | 39,430 | 137,781 | 25,299 | 3,499 | 88,521,201 |
| AIM-III [7] | 1,144,536 | 132,785 | 464,373 | 88,395 | 8,583 | 758,694,285 |
| AIM-V [7] | 1,486,100 | 157,588 | 551,124 | 108,072 | 16,857 | 1,821,769,704 |
| AIM2-I (our) | 1,921,984 | 218,768 | 765,450 | 119,763 | 11,861 | 1,420,508,943 |
| AIM2-III (our) | 4,405,986 | 551,124 | 1,928,934 | 300,436 | 41,045 | 12,331,354,575 |
| AIM2-V (our) | 7,674,035 | 866,052 | 3,031,182 | 479,801 | 69,072 | 33,140,814,672 |

Next, we estimate the Grover key search cost of AIM2(see Table 6) and compare it with that of AES variants(see Table 7). For AES, we use the NIST estimates [13,14], derived from the analyses of Grassl et al. [3] and Jaques et al. [8].

In our estimation, the diffusion operator is ignored and only the oracle cost is considered [8,5]. The Grover oracle consists of (1) the AIM quantum circuit for encryption, (2) an $n$-controlled NOT gate to compare the computed ciphertext with a known ciphertext (cost: $(32n - 64)$ $T$ gates, where $n$ is the ciphertext size), and (3) the inverse of the AIM quantum circuit for uncomputation. The total oracle cost is therefore

$$\left\lfloor \frac{\pi}{4}\sqrt{2^k} \right\rfloor \times \Big( (32 \cdot n - 64)T \text{ gates} + 2 \times \text{Table 5} \Big).$$

The number of Grover iterations is $\lfloor \pi/4 \cdot \sqrt{2^k} \rfloor$, and the additional decision qubit required for ciphertext verification increases the qubit count by only one, since the iterations are sequential.

We then compare the Grover costs of AES and AIM2 to assess the post-quantum security of AIM2. Table 7 presents the results. A direct comparison with the original NIST estimates [13] suggests that AIM2-I, -III, and -V do not meet the Level-1, -3, and -5 security levels, respectively. This is because AES circuits are extremely costly [3], and the NIST estimates were conservatively set. Later, NIST revised the AES Grover cost estimates [14] based on the work of Jaques

Table 6: Costs of the Grover's key search for AIM2

| Cipher | Total gates | Total depth | Cost (complexity) | #Qubit | $FD \times M$ |
|---|---|---|---|---|---|
| AIM2-I | $1.09 \times 2^{86}$ | $1.14 \times 2^{78}$ | $1.24 \times 2^{164}$ | 119,764 | $1.04 \times 2^{95}$ |
| AIM2-III | $1.29 \times 2^{119}$ | $1.97 \times 2^{111}$ | $1.27 \times 2^{231}$ | 300,435 | $1.13 \times 2^{130}$ |
| AIM2-V | $1.08 \times 2^{152}$ | $1.66 \times 2^{144}$ | $1.79 \times 2^{296}$ | 479,801 | $1.52 \times 2^{163}$ |

et al. [8], and we adopt these updated values in our evaluation. Although some studies have argued that the revised estimates may still be underestimated [5], we conclude that AIM2-I, -III, and -V meet the Level-1, -3, and -5 security levels when compared against the updated AES estimates.

Table 7: Comparison of the Grover's key search costs

| Post-quantum Security | NIST'16 [13] (based on [3]) | NIST'22 [14] (based on [8]) | AIM2 | | |
|---|---|---|---|---|---|
| | | | -I | -III | -V |
| Level-1 (AES-128) | $2^{170}$ | $2^{157}$ | $2^{164}$ | | |
| Level-3 (AES-192) | $2^{233}$ | $2^{221}$ | | $2^{231}$ | |
| Level-5 (AES-256) | $2^{298}$ | $2^{285}$ | | | $2^{296}$ |

## 7 Conclusion

In this work, we proposed quantum circuit implementations of AIM2. We performed optimizations to reduce the circuit depth and introduced a qubit-reuse technique. For all AIM2 variants (AIM-I, AIM-III, AIM-V), we designed quantum circuits for the internal operations, including $\mathsf{Mer}^{-1}$, the linear layer, $\mathsf{Mer}$, and feed-forward. Compared to the previous version of AIM, AIM2 requires significantly more quantum resources, mainly because the $\mathsf{Mer}^{-1}$ operation is applied before the linear layer. Based on the proposed circuits, we estimate the cost of Grover's algorithm for key search and confirm post-quantum security levels. As a result, AIM2-I, -III, and -V meet Level-1, -3, and -5 security levels, respectively, compared to the updated AES cost estimates by NIST. These results demonstrate that AIM2 provides strong post-quantum security.

## 8 Acknowledgment

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2019-II190033, Study on Quantum Security Evaluation

of Cryptography based on Computational Quantum Complexity, 10%) and this work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. RS-2025-23523436, 90%).

## References

1. Amy, M., Maslov, D., Mosca, M., Roetteler, M.: A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **32**(6), 818–830 (2013)
2. Cheung, D., Maslov, D., Mathew, J., Pradhan, D.K.: On the design and optimization of a quantum polynomial-time attack on elliptic curve cryptography. In: Workshop on Quantum Computation, Communication, and Cryptography. pp. 96–104. Springer (2008)
3. Grassl, M., Langenberg, B., Roetteler, M., Steinwandt, R.: Applying Grover's algorithm to AES: quantum resource estimates. In: International Workshop on Post-Quantum Cryptography. pp. 29–43. Springer (2016)
4. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the twenty-eighth annual ACM symposium on Theory of computing. pp. 212–219 (1996)
5. Jang, K., Baksi, A., Kim, H., Song, G., Seo, H., Chattopadhyay, A.: Quantum analysis of AES. Cryptology ePrint Archive (2022)
6. Jang, K., Kim, W., Lim, S., Kang, Y., Yang, Y., Seo, H.: Optimized implementation of quantum binary field multiplication with toffoli depth one. In: International Conference on Information Security Applications. pp. 251–264. Springer (2022)
7. Jang, K., Oh, Y., Kim, H., Seo, H.: Quantum implementation of AIM: aiming for low-depth. Applied Sciences **14**(7), 2824 (2024)
8. Jaques, S., Naehrig, M., Roetteler, M., Virdia, F.: Implementing grover oracles for quantum key search on AES and LowMC. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 280–310. Springer (2020)
9. Kepley, S., Steinwandt, R.: Quantum circuits for $\mathbb{F}_{2^n}$-multiplication with sub-quadratic gate count. Quantum Information Processing **14**(7), 2373–2386 (2015)
10. Kim, S., Ha, J., Son, M., Lee, B., Moon, D., Lee, J., Lee, S., Kwon, J., Cho, J., Yoon, H., et al.: The aimer signature scheme. Tech. rep., Technical report. NIST (2023)
11. Lee, J., Cho, A.J., Kim, S., Kwon, J., Lee, B., Lee, J., Lee, S., Moon, D., Son, M., Yoon, H., et al.: The aimer signature scheme
12. Nielsen, M.A., Chuang, I.L.: Quantum computation and quantum information. Cambridge university press (2010)
13. NIST.: Submission requirements and evaluation criteria for the post-quantum cryptography standardization process (2016), https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf
14. NIST.: Call for additional digital signature schemes for the post-quantum cryptography standardization process (2022), https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf
15. Shor, P.W.: Algorithms for quantum computation: discrete logarithms and factoring. In: Proceedings 35th annual symposium on foundations of computer science. pp. 124–134. Ieee (1994)

16. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM review **41**(2), 303–332 (1999)
17. Van Hoof, I.: Space-efficient quantum multiplication of polynomials for binary finite fields with sub-quadratic Toffoli gate count. arXiv preprint arXiv:1910.02849 (2019)